



UNIVERSIDADE FEDERAL DO CEARÁ
GRADUAÇÃO EM CIÊNCIA DA COMPUTAÇÃO

GUSTAVO CAMPELO DE SOUSA
ANTONIO JORGE NOBRE DA COSTA
FRANCISCO JEFFERSON LIMA DE MIRANDA
JOÃO MATHEUS ANTUNES CAVALCANTE
LAÉRCIO LEVI SILVA DE MELO

TRABALHO DE PROJETO E ANÁLISE DE ALGORITMOS

CRATEÚS

2024

TRABALHO DE PROJETO E ANÁLISE DE ALGORITMOS

Relatório do trabalho do primeiro módulo da disciplina de Projeto e Análise de Algoritmos, tratando da implementação de algoritmos de ordenação, análise de tempo e análise assintótica, além da implementação de um algoritmo híbrido e sua comparação de desempenho.

Professor: Prof. Dr. Rafael Martins Barros.

RESUMO

Relatório das atividades realizadas durante o período de dezembro/2024 e início de janeiro/2024 relacionadas ao entendimento e implementação dos principais algoritmos de ordenação, além de suas medições de tempo, análise de complexidade e estudo para implementação de um algoritmo híbrido levando em conta os outros algoritmos de ordenação. O relatório contém a descrição de tudo que foi feito, em casos de implementação, análise da complexidade teórica, detalhes de comportamento de código e sobre como foi feita a medição do tempo, contendo também o relato sobre o algoritmo híbrido e sua medição de tempo.

SUMÁRIO

1	INTRODUÇÃO	5
2	SOBRE A IMPLEMENTAÇÃO DOS ALGORITMOS.....	6
2.1	Geração do vetor aleatório e vetor ordenado.....	6
2.2	Implementação do Bubble Sort.....	6
2.2.1	Complexidade do Bubble Sort.....	7
2.3	Implementação do Insertion Sort.....	9
2.3.1	Complexidade do Insertion Sort.....	10
2.4	Implementação do Merge Sort.....	12
2.4.1	Complexidade do Merge Sort.....	13
2.5	Implementação do Quick Sort.....	14
2.5.1	Complexidade do Quick Sort.....	16
3	SOBRE A GERAÇÃO DOS GRÁFICOS DE MEDIÇÃO.....	17
4	SOBRE A ANÁLISE DOS RESULTADOS.....	20
5	SOBRE O ALGORITMO HÍBRIDO.....	23
6	COMPARAÇÃO ENTRE OS ALGORITMOS PUROS E O HÍBRIDO.....	24
7	CONCLUSÃO.....	26

1 INTRODUÇÃO

Os algoritmos de ordenação estão para as estruturas de dados, como um trator para uma escavação de terrenos, e como essa analogia se concretiza? Suponhamos que haja uma lista preenchida com 1000 números inteiros não repetidos, de maneira aleatória, paralelamente ao que temos um terreno para escavar, e temos o “olhômetro” e uma pá, respectivamente. A partir disso é simples analisar que a carga de trabalho e o tempo necessário para ordenar tal lista de elementos e para escavar esse terreno tende a levar um tempo excepcionalmente alto. Porém, supondo que temos um algoritmo que faça a ordenação e um trator que escave o terreno, a carga de trabalho reduzir-se-á de uma forma satisfatória, e por vezes instantânea, a depender do tamanho da lista e do terreno.

A partir dessa analogia, se tem como essencial o entendimento desses algoritmos no aprendizado para resolução de problemas em estruturas de dados, e com o auxílio de autores renomados como Thomas Cormen, a fim de comprovar as observações feitas e anotadas durante a realização desse trabalho, e com a ajuda em seus pseudocódigos, análises assintóticas dos algoritmos, funções de recorrência e por fim a complexidade assintótica dos algoritmos.

Esse trabalho tem como foco lembrar os principais algoritmos de ordenação conhecidos e ensinados nos semestres iniciais, entender seu funcionamento e a notação de complexidade obtida para eles em seus melhores e piores casos, aplicando-os para diferentes tamanhos de entrada de dados e observando seu comportamento. Além disso, será medido o tempo de execução de cada algoritmo, principalmente para as entradas de maior ordem, e com isso verificar se o comportamento prático está de acordo com a análise assintótica teórica. Os algoritmos de ordenação foram implementados utilizando da linguagem Python, com a ajuda de alguns recursos pertencentes à linguagem, como o método “extend”, que adiciona algum elemento restante ao final de uma lista. Foram utilizadas bibliotecas para a geração do vetor aleatório, para a medição do tempo de cada algoritmo de ordenação e para salvar o vetor em um arquivo de texto.

Será visto diversas vezes a utilização das palavras array e lista, onde ambas têm o mesmo significado. Não houve padronização desses nomes pois acreditamos que não haveria confusão de nomes, e cada membro do grupo colaborou à maneira para a construção dos capítulos.

Portanto, através desses fatores, somados ao bom trabalho em equipe, tornaram possíveis toda as tentativas de capricho e grande aprendizado obtido durante todo esse processo.

2 SOBRE A IMPLEMENTAÇÃO DOS ALGORITMOS

Os algoritmos implementados foram o Bubble Sort, Insertion Sort, Merge Sort e o Quick Sort, onde todos estes recebem um vetor com “n” posições, e através dele, estando preenchido em uma ordem completamente aleatória, têm os algoritmos aplicados e ao final de cada um é retornado o vetor ordenado corretamente. Será descrito a implementação de todos os arquivos e como cada algoritmo funciona.

2.1 Geração do vetor aleatório e vetor ordenado

A geração do vetor aleatório se deu através da grande gama de possibilidades de utilização da biblioteca “random”, e como foi preferido evitar problemas de repetição de valores inteiros no vetor, foi declarado inicialmente um vetor vazio, e esse vetor receberia um gerador de variáveis aleatórias vindas da biblioteca, instanciada com a função “sample”, que nada mais faz do que escolher k elementos aleatórios de uma sequência definida em seus parâmetros, ou seja, “random.sample(range(1000)), arraySize)”, será gerado os números aleatórios no intervalo escolhido, e o segundo parâmetro trata de quantos números será selecionado no intervalo, nesse caso no tamanho do array.

Após a geração do array com números aleatórios, foi decidido salvá-lo em um arquivo de texto no formato de lista, mesmo já o estando, pois ficaria mais simples de resgatá-lo para aplicá-lo aos algoritmos. Para isso, o array será salvo na pasta “arraysNotSorted”, no arquivo “array.txt”, e com isso tenta executar a “saveArray”, que possui a função de salvamento do array, e escreve o vetor gerado dentro desse arquivo através do formato de string, assim salvando o array corretamente no formato de lista, porém como um texto [0, 1, 2,..., n].

Foi realizado também a implementação de um arquivo que gera um vetor ordenado. Na verdade, simplesmente trata-se de uma variável que chama a classe “list” que vai gerar uma sequência do tamanho do parâmetro passado pelo usuário, são duas linhas de código. E feito isso, esse array é salvo em um arquivo, da mesma forma que o aleatório.

Essa implementação se deu pelo interesse em analisar também os melhores casos dos algoritmos de ordenação, principalmente os não recursivos como Bubble Sort e Insertion Sort, que na teoria tem complexidades diferentes do pior para o melhor caso. E isso também foi analisado e relatado neste relatório, mais especificamente no capítulo quatro.

2.2 Implementação do Bubble Sort

O Bubble Sort é um dos algoritmos de ordenação mais simples de se entender e implementar, pois ele simplesmente compara o elemento atual com o elemento seguinte, e se esse for maior, será realizada a troca, e com isso é percorrido toda a extensão da lista, e depois cada elemento da lista, o que nos traz dois laços de repetição, onde uma realiza operações do

tamanho da lista, e dentro de cada operação, são feitas mais operações partindo do primeiro elemento da lista até o penúltimo elemento ainda não ordenado da lista.

Seguindo a explicação de cada linha do algoritmo, haverá uma tabela com o custo de cada operação, com base no livro Algoritmos, do Thomas Cormen. O algoritmo começa definindo o tamanho da lista, que é do tamanho da lista em si e logo após é iniciado um laço de repetição que será executado durante toda a extensão da lista, e com isso haverá outro laço aninhado com o primeiro, que após cada iteração após cada iteração completa do laço externo, o maior elemento restante na lista vai "borbulhar" para a posição correta no final da lista.

Na primeira iteração ($i = 0$), j percorre de 0 até tamanhoLista - 1.

Na segunda iteração ($i = 1$), j percorre de 0 até tamanhoLista - 2.

E dentro do segundo laço, é verificado se o elemento j da lista é maior que o elemento $j+1$, ou seja, o próximo elemento é maior, e caso for, esses 2 elementos serão trocados de posição, partindo para a posição correta, pelo menos com relação à anterior.

2.2.1 Complexidade do Bubble Sort

A seguir é exibida uma tabela com o custo de cada linha de acordo com o que temos no arquivo de implementação realizado no VSCode, ou melhor dizendo, a linha 5 corresponde à linha onde começa a implementação, e logo após, será explicitado sobre a complexidade do Bubble Sort:

Linha	Custo	Veze
5	c5	1
6	c6	n
7	c7	$\sum_{i=0}^{n-1} (n-1-i)$ ou $n(n-1)/2$ (laço que percorre $n-i-1$ a cada iteração)
8	c8	$\sum_{i=0}^{n-2} (n-1-i)$ ou $n(n-1)/2$ (todas as comparações são feitas)
9	c9	k, onde k é o número de trocas realizadas no vetor

O tempo de execução do algoritmo é a soma dos tempos de execução para cada instrução executada; uma instrução que demanda “ci” passos para ser executada e é executada n vezes contribuirá com $c_i \cdot n$ para o tempo de execução total. Para calcular $T(n)$, o tempo de execução do Bubble Sort de uma entrada de n valores, somamos os produtos das colunas custo e vezes, onde para o pior caso a quantidade de trocas é no máximo o a quantidade de iterações realizadas, logo nosso $k \leq n(n-1/2)$. Enfim, para o pior caso temos a fórmula:

$T(n) = 1 + c_6(n) + c_7(\sum_{i=0}^{n-1} (n - i - 1)) + c_8(\frac{n-1}{2}) + c_9 * (\frac{n-1}{2})$, e ao expandi-la, obtemos a fórmula $T(n) = 1 + c_6(n) + c_7(\frac{n(n-1)}{2}) + (c_8+c_9) * (\frac{n-1}{2})$

Esse é o pior caso, porque considera que a lista está organizada de maneira decrescente, ou mesmo de forma desorganizada (caso médio), ou seja, ao ter a lista organizada de uma maneira tão ruim assim, as trocas de valores irão ocorrer em todas as iterações do loop aninhado, ou em quase todas.

Porém, no melhor caso, onde temos a lista ordenada, ou uma lista de tamanho 1, não irá ocorrer troca de valores, já que o vetor já está ordenado, porém mesmo assim haverá a execução do primeiro e do segundo for, e esse segundo for não será preciso representar como somatório, já que fará apenas uma execução, trazendo assim a fórmula:

$T(n) = 1 + c_6(n) + c_7(\frac{n-1}{2}) + c_8(\frac{n-1}{2}) + c_9 * (\frac{n-1}{2})$, e ao expandir, obtemos:

$$T(n) = (c_6 + \frac{c_7}{2} + \frac{c_8}{2} + \frac{c_9}{2}) * n - (\frac{c_7}{2} + \frac{c_8}{2} + \frac{c_9}{2}) + 1$$

Ao desenvolver as fórmulas, levando em conta o caso em que a lista está organizada de maneira decrescente, será realizado o máximo possível de comparações e trocas entre os elementos do vetor, e com isso, já será realizada o máximo para o loop externo, que rodará “n” vezes; já no loop interno, para cada “i”, será rodado “n-1-1” vezes, e no pior dos casos, a condição do elemento atual ser maior que o subsequente sempre será verdadeira, então todas as comparações possíveis irão ser realizadas, tanto no melhor como no pior caso. Porém, somente no pior caso é que a notação da quantidade de comparações será igual ao número de trocas, o que não significa que a quantidade que cada uma realizará será igual, mas sim sua notação de valor máximo! Ou seja:

$$\text{Número de comparações} = \text{Número de trocas} = (n-1/2)$$

O primeiro termo é o custo por cada comparação feita, e esta sempre será a do pior caso, pois mesmo tendo um vetor de tamanho 2, irá ser feita a quantidade máxima de comparações dada pela notação. E quanto ao segundo termo, que é a quantidade de trocas, ela só será relevante em casos que serão necessárias muitas trocas, e somente no pior caso é que será realizada uma quantidade máxima de trocas, entretanto a partir de um limite estrito já é possível considerar essa notação. E através de toda a teoria existente quanto ao Bubble Sort, e expandindo a equação, por meio dos termos de maior dominância, é evidente que teremos no pior caso, uma notação tanto $O(n^2)$ quanto $\theta(n^2)$, porém no melhor caso pode-se expressar o Bubble Sort com a notação $\Omega(n)$, ou mesmo com $\theta(n)$, pois é evidente que será feita pelo menos uma quantidade “n” de operações proporcionais à comparação, que é do tamanho do vetor, porém não será feita nenhuma operação de troca, logo mantendo a análise assintótica de maneira linear.

2.3 Implementação do Insertion Sort

O Insertion Sort, ou Ordenação Por Inserção, é um algoritmo de ordenação eficiente para entradas de tamanho pequeno, assim como também é o Bubble Sort, entretanto, ao aumentarmos o tamanho da entrada, suas semelhanças de lentidão com o Bubble começam a aparecer. Ele funciona como se tivéssemos um baralho e fôssemos pegando uma carta por vez, e a cada vez que víssemos a carta selecionada, colocaríamos na posição correta, e isso ocorreria para cada carta retirada do montante. Mas levando a um nível mais baixo, a cada carta selecionada, compararíamos ela com cada uma das cartas que já tivéssemos na mão esquerda (que contém a sequência), e essa comparação se daria da direita para a esquerda. Em todas as vezes, as cartas que segurariamos na mão esquerda estariam ordenadas, e essas cartas seriam as que estavam na parte superior da pilha sobre a mesa.

A implementação se deu a partir da definição do tamanho da lista, que é igual ao tamanho da lista em si. Há um laço de repetição que percorre todo o tamanho do vetor (arranjo). Em seguida, é declarada a variável chave, que armazena o valor atual do array na posição j, sendo o elemento que será inserido na parte ordenada do array.

Depois, a variável i é inicializada com uma posição anterior a j, para indicar o índice do último elemento da parte ordenada do array. Após isso, um while é usado para mover os elementos da lista que são maiores que a chave uma posição para a direita. O loop continua enquanto i for maior ou igual a 0 e chave for menor que lista[i]. Com isso, o elemento é movido para a direita, decrementando i para que a verificação possa continuar.

Quando o while termina, a chave é inserida na posição correta em $\text{lista}[i + 1]$. O segredo está na combinação do for com o while. O for só termina quando todo o array estiver ordenado, e o while continua executando enquanto for possível iterar pelo array.

Por exemplo, ao pegar a lista [23, 1, 10, 5, 2], começamos pelo elemento 1, pois assume-se que o elemento 0 já está ordenado. Em seguida, a comparação de 1 com 23 é feita. Se a comparação for válida, a posição é trocada e o valor da chave é incrementado (a chave é o próximo elemento não ordenado a ser comparado com os elementos já ordenados).

A chave, após a comparação, é inserida na posição correta na parte ordenada. Em seguida, a chave passa a ser o próximo elemento da lista a ser comparado, continuando até percorrer todo o array. A quantidade total de chaves é $n-1$ (tamanho do array menos o primeiro elemento).

2.3.1 Complexidade do Insertion Sort

A seguir é exibida uma tabela com o custo de cada linha de acordo com o que temos no arquivo de implementação realizado no VSCode, da mesma maneira com o que foi feito no Bubble Sort, e logo após, será explicitado sobre a complexidade do atual Insertion Sort:

Linha	Custo	Veze
5	c5	1
6	c6	n (execução do for)
7	c7	n-1 (atribuição inicial da chave)
8	c8	n-1
10	c10	$\sum_{j=1}^n (tj)$ execução do while para mover os elementos da lista que são maiores que a chave
11	c11	$\sum_{j=1}^n (tj - 1)$ execução da troca dos elementos
12	c12	$\sum_{j=1}^n (tj - 1)$ Move i uma posição à esquerda para continuar comparando com o próximo elemento à esquerda.
14	c14	n-1 inserir a chave na posição correta
16	c16	1

O tempo de execução do algoritmo é a soma dos tempos de execução para cada instrução executada; uma instrução que demanda “ci” passos para ser executada e é executada n vezes contribuirá com $ci \cdot n$ para o tempo de execução total. Para calcular $T(n)$, o tempo de

execução de INSERTION-SORT de uma entrada de n valores, somamos os produtos das colunas custo e vezes, obtendo:

$$T(n) = c5*1 + c6*n + c7(n-1) + c8(n-1) + c10 * \sum_{j=1}^n(tj) + c11* \sum_{j=1}^n(tj - 1) + c12*(\sum_{j=1}^n(tj - 1) + c14*(n-1) + c16*1$$

Esse é o caso em que todas as operações são consideradas, visto que acaba considerando que será percorrido cada elemento da lista, iniciando pelo segundo, e o inserindo na posição correta na parte ordenada, e no pior caso, cada elemento vai precisar ser comparado com todos os elementos anteriores para ser movido até o início da lista. E quanto ao número de comparações, para cada elemento j , o número de comparações é $j-1$, visto que esse elemento vai ser comparado com todos os elementos já ordenados. Ao levar em conta os somatórios presentes na função de custo, será necessário resolvê-los, que com a ajuda do Cormen, obtemos, porém para resolvê-los seria necessário realizar as operações de uma progressão aritmética:

$$\sum_{j=1}^n(j) = \frac{n(n-1)}{2} - 1$$

$$\sum_{j=1}^n(j - 1) = \frac{n(n-1)}{2}$$

A partir disso, substituindo nos termos, na função de custo, teremos:

$$T(n) = c5*1 + c6*n + c7(n-1) + c8(n-1) + c10 *(\frac{n(n-1)}{2} - 1) + c11(\frac{c10}{2} + \frac{c11}{2} + \frac{c12}{2})n^2 + (c6 + c7 + c8 + \frac{c10}{2} - \frac{c11}{2} - \frac{c12}{2} + c14)n^1 - (c7 + c8 + c10 + c14)$$

Ao desenvolver a expressão, obteremos por fim uma expressão com o elemento de maior dominância sendo quadrático, podendo ser expressada como $an^2 + bn + c$ para constantes a , b e c que, mais uma vez, dependem dos custos de instrução c_i ; portanto, ele é uma função quadrática de n . Em semelhança com o Bubble Sort, uma das etapas será executada na totalidade do algoritmo. No caso do Insertion, é o tempo necessário para ordenar os $n-1$ primeiros elementos, e o segundo subproblema se concentra no tempo necessário para inserir o novo elemento na posição correta, com comparações e deslocamentos, que somente no pior caso é que será realizada completamente. E através de toda a teoria existente quanto ao Insertion Sort, e resolvendo a função de recorrência, é evidente que teremos no pior caso, uma notação tanto $O = (n^2)$ quanto $\theta = (n^2)$, porém no melhor caso pode-se expressar o Insertion Sort com a notação $\Omega = (n)$, ou mesmo com $\theta = (n)$,

$$T(n) = c6*n + c7(n-1) + c10(n-1) + c11(n-1) + c14(n-1), \text{ que é igual a}$$

$$T(n) = (c6 + c7 + c10 + c11 + c14)n$$

Aqui, é evidente que será feita pelo menos uma quantidade “n” de operações, que é do tamanho do vetor, porém não será feita nenhuma operação de troca ao longo da execução, logo mantendo a análise assintótica de maneira linear.

2.4 Implementação do Merge Sort

O Merge Sort é um algoritmo de ordenação que funciona diferente dos dois anteriores, pois este é recursivo, logo é um algoritmo que chama a si mesmo para lidar com subproblemas relacionados com o próprio algoritmo. Além do que é de uma abordagem de divisão e conquista, onde um problema é desmembrado em outros menores do mesmo tipo e após isso são resolvidos e por fim combinados para a solução pronta. Outro fator muito interessante é que sua função de recorrência é mais simples de se denominar, visto que a divisão e conquista implica os passos que devem ser realizados dentro do algoritmo.

A implementação se deu por meio da definição de duas funções: mergeSort e merge, onde a primeira realiza a parte de divisão e conquista, enquanto a outra combina os problemas resolvidos pela função anterior.

A função de divisão e conquista se inicia com o caso base, do qual é também o ponto de parada da recursão, então se o tamanho do array/lista for menor ou igual a 1, a lista já está ordenada, do contrário, a etapa de divisão será iniciada, dividindo a lista pela metade, por meio de uma divisão inteira, logo após, inserimos a primeira metade em uma variável, e a segunda metade em outra, indicando a da direita e da esquerda. A partir disso, chama-se recursivamente a função de ordenação para ordenar os dois lados, retornando a função merge que fará a combinação das partes ordenadas.

A função de combinação funciona declarando um array vazio que é o local onde será armazenado o resultado final da combinação, as variáveis do laço de repetição serão iniciadas com 0. Feito isso, haverá um laço de repetição do qual enquanto i for menor do que o lado esquerdo e j for menor que o lado direito, esse laço irá se repetir; e se o valor da esquerda for menor que o da direita, este irá ser adicionado ao resultado, e avançará para a próxima posição, senão será adicionado ao resultado e avançará para a próxima posição da direita.

Ao final os elementos restantes da lista da direita e da esquerda serão adicionados a partir de seus índices i e j, na lista da esquerda e direita, respectivamente, ou seja, após o while onde será comparado os elementos da esquerda e direita, todos os elementos restantes ainda não comparados, serão adicionados à lista final.

Um exemplo da implementação é uma lista [3, 1, 4, 2, 5], esta será dividida em duas listas, [3, 1] na metade esquerda e [4, 2, 5] na metade direita. A chamada recursiva irá ocorrer até dividir a lista em partes unitárias, a começar pelo lado esquerdo, ficando [3] e [1], e arrays

unitários, por definição, já estão ordenados. O mesmo é feito com o lado direito, a divisão até tamanhos unitários, depois a resolução dos problemas, e a combinação dos sub-arrays. E por fim sobra-se novamente as duas listas ordenadas para ser realizado a combinação, que resulta no retorno final do array corretamente ordenado.

2.4.1 Complexidade do Merge Sort

Para explicar sobre a complexidade do Merge Sort, será utilizado uma maneira diferente do que a tabela de custo dos algoritmos anteriores. Cormen tem um tópico próprio para análise de exemplos de divisão e conquista, do qual é dito que ao haver chamadas recursivas, podemos expressar seu tempo de execução por uma equação de recorrência, diferentemente de algoritmos iterativos, do qual sua análise é mais direta. Pensando de maneira mais direta, a equação tende a ser composta pela etapa de divisão dos problemas menores, a etapa de conquista desses problemas e a combinação dos problemas resolvidos. A equação considera $T(n)$ como o tempo para um problema de tamanho “ n ”, e se esse problema for pequeno o suficiente, $n \leq c$ para alguma constante c , a solução direta demorará um tempo constante, que representamos por $\theta(1)$. Nesse caso os problemas são subdivididos em 2 subproblemas de tamanho $\frac{n}{2}$ e os resolve nesse tempo, e de acordo com o Cormen: “Se é levado o tempo $aT(n/b)$ para resolver um número a desses problemas, e o algoritmo levar o tempo $D(n)$ para dividir o problema em subproblemas e o tempo $C(n)$ para combinar as soluções dadas aos subproblemas na solução para o problema original”. Com essa citação, será obtida a recorrência no formato a seguir:

$$T(n) = \begin{cases} \theta(1), & \text{se } n \leq c \\ aT\left(\frac{n}{b}\right) + D(n) + C(n), & \text{caso contrário} \end{cases}$$

O pensamento mais simples do qual o grupo entendeu é que esse formato de recorrência se aplica para a maioria dos algoritmos de divisão e conquista, e tratando do exemplo do Merge Sort, temos as etapas e seus custos:

- Divisão: Aqui é calculado o ponto médio do array, o que é algo constante, logo pode ser representado por $T(n) = \theta(1)$.
- Conquista: Aqui os subproblemas são resolvidos, a cada etapa eles tem 2 subproblemas de tamanho $\frac{n}{2}$, contribuindo com $2T\left(\frac{n}{2}\right)$ para o tempo.
- Combinação: Aqui no array de n elementos, essa etapa leva um tempo linear, comparando cada elemento e adicionando apenas uma vez, no máximo igual ao tamanho do array $\theta(n)$.

Substituindo os termos da definição pelos obtidos na análise do Merge Sort, temos por fim a seguinte equação de recorrência: $T(n) = 2T\left(\frac{n}{2}\right) + \theta(n)$. O termo $D(n)$ se viu ignorado nessa equação devido a sua soma com a função linear, que resulta em uma função linear, logo não há necessidade de colocá-la na equação de recorrência. Assim como aprendido em sala, há a possibilidade resolvermos pelos métodos da substituição, árvore de recursão ou pelo teorema mestre. Afim de economizar tempo, segue a resolução pelo teorema mestre, já que esta recorrência está no formato $T(n) = aT\left(\frac{n}{b}\right) + F(n)$, logo devemos separar os termos, onde $a = 2$, $b = 2$ e $F(n) = \theta(n)$, que também pode ser reescrito como $c * n$.

Pelo método mestre, há de ser feita a operação $n^{\log_b a}$, que é $n^{\log_2 2}$. E ao resolver essa operação, temos n^1 , e $n^1 = F(n)$, o que nos traz ao segundo caso do método mestre, que diz se $F(n) = \theta(n^{\log_b a})$, então $T(n) = (n^{\log_b a} \log n)$, portanto, temos que $T(n) = (n^{\log_2 2} \log n)$, ou melhor $T(n) = (n \log n)$, visto que base do logaritmo para uma análise assintótica é irrelevante, logo temos que a complexidade do Merge Sort é $\theta(n \log n)$, ou seja, tanto para o pior caso quanto para seu melhor caso, seu tempo de execução está no limite estrito, realizando a mesma quantidade de operações para todos os casos.

2.5 Implementação do Quick Sort

O Quick Sort é um algoritmo de ordenação recursivo, seguindo os passos típicos de divisão e conquista, dividindo o problema em partes menores até alcançar subproblemas simples, e é amplamente usado devido à sua eficiência no caso médio, tendo complexidade igual ao Merge Sort, entretanto para o pior caso seu tempo de execução é $\theta(n^2)$. Apesar de ser complexo, o livro do Cormen explica seu funcionamento de forma clara. Seu funcionamento depende de alguns fatores, que será relatado aqui de acordo com os testes feitos na implementação.

Sua implementação se deu pelo uso de duas funções, uma de partição e a outra da utilização da recursão, na função da recursão, há o caso base, do qual caso a lista contenha apenas 1 elemento, irá estar ordenada, por definição. Mas falando primeiro da repartição, que será passado o array, o início e o final deste. O elemento do meio da lista é escolhido como pivô, pois essa escolha tende a melhorar o desempenho do algoritmo. Feito isso, o pivô é movido para o final para facilitar as comparações na partição, e depois disso o índice do menor elemento é o elemento inicial menos um, e será utilizado para rastrear a posição do último elemento menor ou igual ao pivô. Será iniciado um laço de repetição partindo do inicial até o último menos um.

Com isso, há a condição dentro do for, de que se o índice atual do array for menor ou igual ao pivô, o índice do menor elemento será incrementado para o próximo elemento, trocando o índice do menor elemento com o índice atual. Agora, após o for, todos os elementos que são menores ou iguais ao pivô foram movidos para o início do array, até a posição do índice de último elemento menor ou igual ao pivô.

Melhor explicando, suponha que há o array [3, 6, 8, 10, 1, 2, 1] e vamos particionar usando o elemento do meio como pivô. Ele irá ser movido para o final da lista, trocando de lugar com o último elemento, ficando [3, 6, 8, 1, 1, 2, 10]. Com isso, há a condição dentro do for de que, se o elemento atual for menor ou igual ao pivô, o índice do menor elemento será incrementado, e os elementos trocados, se necessário. Após o laço, o pivô será posicionado corretamente na lista, terminando na posição 6, todos os elementos menores ou iguais ao pivô foram movidos para sua esquerda.

Partindo para a função quickSort, passaremos também o array, o início e o fim do array como parâmetros, definiremos o caso base de uma função recursiva. E isso se concluindo, será atribuída ao índice da partição para reorganizar os elementos da sublista do início ao final ao redor do pivô, e essa função de partição vai retornar o índice do pivô na posição correta. E feito isso, será chamada recursivamente a função de quick sort para ordenar os elementos à esquerda do pivô e os elementos à direita do pivô, continuando a executar até as sublistas terem tamanho 0 ou 1.

Utilizando de um exemplo [7, 2, 1, 6, 8, 5, 3, 4], na primeira execução, temos os parâmetros start = 0 e end = 7, sendo o array completo. O pivô é escolhido, irá ser colocado na última posição e trocado com o elemento que antes estava lá e o array será particionado. Seguem as comparações e trocas após o pivô ter ido para a última posição [7, 2, 1, 4, 8, 5, 3, 6]:

- Elemento: $7 > 6$, logo não haverá troca, smaller_index = -1
- Elemento $2 \leq 6$, troca o 7 com 2, smaller_index = 0
- Elemento $1 \leq 6$, troca 7 com 1, smaller_index = 1
- Elemento $4 \leq 6$, troca 7 com 4, smaller_index = 2
- Elemento $8 > 6$, logo não haverá troca, smaller_index = 2
- Elemento $5 \leq 6$, troca o 7 com 5, smaller_index = 3
- Elemento $3 \leq 6$, troca o 8 com 3, smaller_index = 4

Após o loop, o pivô 6 irá para a posição correta, visto que o 7 trocará de lugar com o 7, e o smaller_index será igual a 5. O Resultado [2, 1, 4, 5, 3, 6, 8, 7] é o estado do array após a primeira iteração da partição, temos todos os elementos da esquerda

menores que o pivô e os da direita maiores. O algoritmo continuará recursivamente nas sublistas à esquerda e à direita do pivô, dividindo o array em pedaços cada vez menores até que todas as sublistas estejam ordenadas.

2.5.1 Complexidade do Quick Sort

Tratando da complexidade do Quick Sort, não é tão simples de se tratar quanto dos outros algoritmos de ordenação explicados anteriormente, visto que por meio de alguns detalhes a complexidade por aumentar para algo semelhante ao Bubble Sort, e por outro lado, pode ser igual ao Merge Sort. Porém, como o Quick Sort é um algoritmo de divisão e conquista, ele ainda entra nessa definição, mesmo tendo que considerar alguns detalhes a mais a depender da organização da função de partição. Para a experimentação e o relatório, foram considerados apenas o pior caso e o melhor/médio caso. Segue a descrição das etapas:

- Divisão: subdivide-se o problema de ordenar a lista completa em problemas menores de ordenar sublistas, a escolha do pivô, reorganização dos elementos com base no pivô.
- Conquista: Os problemas menores são resolvidos através da recursão, ordenando as sublistas formadas ao redor do pivô.
- Combinação: A etapa que possui a mesma função da combinação no Merge Sort, combinar as sublistas ordenadas através de uma operação de custo linear.

Sobre como o algoritmo desempenhará, irá depender de como os elementos são escolhidos para a partição e o quão balanceada ela será. Caso ela seja particionada de uma forma balanceada, o pivô irá dividir a lista em duas partes iguais ou algo próximo a isso, para que se tenha recursões mais eficientes. Entretanto, se o particionamento não for balanceado, com a lista dividida de maneira desigual, teremos um grande aumento dos níveis de recursão, e a complexidade irá aumentar significativamente, chegando a algo tão lento quanto ao Insertion ou o Bubble.

O pior caso ocorre, como já dito anteriormente, quando o particionamento produz sublistas de tamanho muito desigual, uma contendo $n-1$ elementos e a outra 0 elementos, por exemplo, com n sendo a quantidade de elementos. E caso esse problema de particionamento ocorra em cada iteração da chamada recursiva, com particionamento custando tempo linear $\theta(n)$, teremos um lado com tamanho 0, que retorna o tempo constante $\theta(1)$, teremos então, a seguinte função de recorrência:

$$T(n) = T(n - 1) + T(0) + \theta(n), \text{ ou}$$

$$T(n) = T(n - 1) + \theta(n).$$

E ao expandir essa recorrência, sendo possível até mesmo utilizando da árvore de recursão, irá ser expandida até chegar ao último nível de custo 1, onde o trabalho em cada nível da árvore vai incluir o custo do somatório e o custo adicional constante, que pode ser ignorado. A ser considerado no somatório, teremos:

$\sum_{i=1}^n \theta(i)$, que aqui, temos um somatório de uma Progressão Aritmética com o primeiro termo $a_1 = 1$ e o último $a_n = n$, junto ao número total de termos n . Com isso temos no final $\frac{n(n-1)}{2}$ que é o mesmo que $\frac{n^2}{2}$. Por fim, temos que seu resultado é $\theta(n^2)$, para o pior caso do algoritmo. Além desse péssimo tempo, ele é repetido quando já se tem um array ordenado corretamente, e em contrapartida dos algoritmos não-recursivos, que executam no tempo $O(n)$.

Relacionado ao melhor caso, é muito mais simples de se analisar e obter o resultado, pois os subproblemas são igualmente divididos, não sendo maior do que $\frac{n}{2}$, já que um é desse tamanho, e o outro subproblema é $\frac{n}{2} - 1$. Aqui temos a recorrência igual ao Merge Sort:

$$T(n) = 2T\left(\frac{n}{2}\right) + \theta(n) \text{ e esta já conhecemos o resultado } T(n) = \theta(n \log n).$$

Há também, o caso “médio”, onde o particionamento é desbalanceado, porém não tanto como no pior caso. Ao supor algo como 9 para 1, onde uma partição tem 9 elementos e a outra apenas 1, nota-se que o desequilíbrio é muito grande, todavia, sua recorrência sendo representada como $T(n) = 99T\left(\frac{n}{100}\right) + T\left(\frac{n}{100}\right) + \theta(n)$, será obtido ainda uma complexidade equivalente a $T(n) = O(n \log n)$.

3 SOBRE A GERAÇÃO DOS GRÁFICOS DE MEDIÇÃO

A medição do tempo dos algoritmos foi separada por arquivo, e em cada um, Bubble, Insertion, Merge e Quick, possuem medição exatamente igual, utilizando dos recursos das bibliotecas do Python Seaborn e Matplotlib, que fornecem grande auxílio e facilidade para “plotar” os gráficos necessários para sua medição.

O funcionamento se dá pelos seguintes passos: o resgate do arquivo que contém o array aleatório, e a partir dessa ação, o conteúdo do documento de texto é retornado para o tipo lista reconhecido. Feito isso foi definido a quantidade de execuções que o algoritmo de ordenação seria executado, para ao final ser realizada a média, porém, antes, é necessária a execução do algoritmo em si, junto da sua medição de tempo, que se inicia logo uma linha antes da execução do algoritmo, e termina quando logo na finalização da ordenação. Com isso, esse

tempo é armazenado e adicionado à uma variável, onde logo depois é feito o cálculo da média e salvo em uma variável com nome intuitivo.

O gráfico é gerado pela biblioteca Seaborn, do tipo “lineplot”, onde passamos como parâmetros cada execução no eixo x, e no eixo y é o tempo de cada um. Além disso ainda podemos definir a cor das linhas do gráfico, independentemente do tipo. Logo após, definimos algumas labels, o título do gráfico e finalmente chamamos a função para o gráfico ser exibido com todas as informações corretamente colocadas, além do tempo médio ser exibido no terminal, que é algo a se atentar.

Ao longo dos testes realizados, foi percebido que algumas execuções demoravam mais de uma hora para acontecer, e a média de tempo era por exemplo de 350.000ms, o que dá quase 6 minutos. É de se pensar que o problema estava na medição de tempo, entretanto, não é isso o que realmente acontece. A leitura de um array com milhões de posições acaba por demorar muito mais do que a execução do algoritmo em si, e a medição de tempo está considerando apenas o início e o fim da execução dele, não as operações de resgate e leitura de arquivo. O único momento em que a leitura do arquivo é considerada na execução do algoritmo, é de onde o algoritmo de ordenação executará, e nesse caso é necessário ler o arquivo. Em virtude disso, a notação assintótica não se altera, pois esse fator de leitura do arquivo tem tempo linear, e todos os algoritmos vistos nesse trabalho tem tempo de execução no pior caso maior do que $O(n)$.

Os testes foram realizados em um Acer Nitro 5, com processador Ryzen 7 4800h com 8 núcleos e 16threads, Memória 24GB DDR4 3200MHz e uma GTX 1650 4GBVRAM.

Abaixo, têm-se as tabelas com todas as execuções realizadas de cada algoritmo, exibindo o tamanho da entrada, a quantidade de testes realizados e o seu tempo correspondente. E no próximo capítulo é feita a análise de cada resultado e se estes condizem com a teoria de cada algoritmo testado.

BUBBLE SORT

TAMANHO/ EXECUÇÕES	10^1	10^2	10^3 e $5*10^3$	10^4 e $5*10^4$	10^5	10^6	10^7
10	0.0ms	0.4537ms	42.0985ms e 1077.2490ms	4138.8130ms e 141949.9910ms	600117.1441ms		
100	0.0132ms	0.3695ms	39.7372ms e 1178.2228ms	4117.7054ms			

INSERTION SORT

TAMANHO/ EXECUÇÕES	10^1	10^2	10^3 e $5*10^3$	10^4 e $5 * 10^4$	10^5	10^6	10^7
10	0.0ms	0.1003ms	13.8114ms e 433.7326ms	1592.2519ms e 55302.7085ms	319201.0710ms		
100	0.0022ms	0.1579ms	15.9038ms e 414.9910ms	1627.2367ms			

MERGE SORT

TAMANHO/ EXECUÇÕES	10^1	10^2	10^3 $5*10^3$	10^4 $5 * 10^4$	10^5	10^6	10^7
10	0.0ms	0.1282 ms	1.6723ms e 10.8118ms	19.5583ms e 132.4569ms	283.1523ms	3646.3478ms	48431.2091ms
100	0.0051ms	0.1048ms	1.8637ms e 10.4502ms	21.0968ms 131.7376ms	279.5156ms	3637.6196ms	

Vale citar uma menção honrosa para um array de 50 milhões de posições que levou 325280.2611ms para ser ordenado, e pouco mais de 1 hora para terminar sua execução, porém, como dito anteriormente, apenas a execução do algoritmo foi considerada.

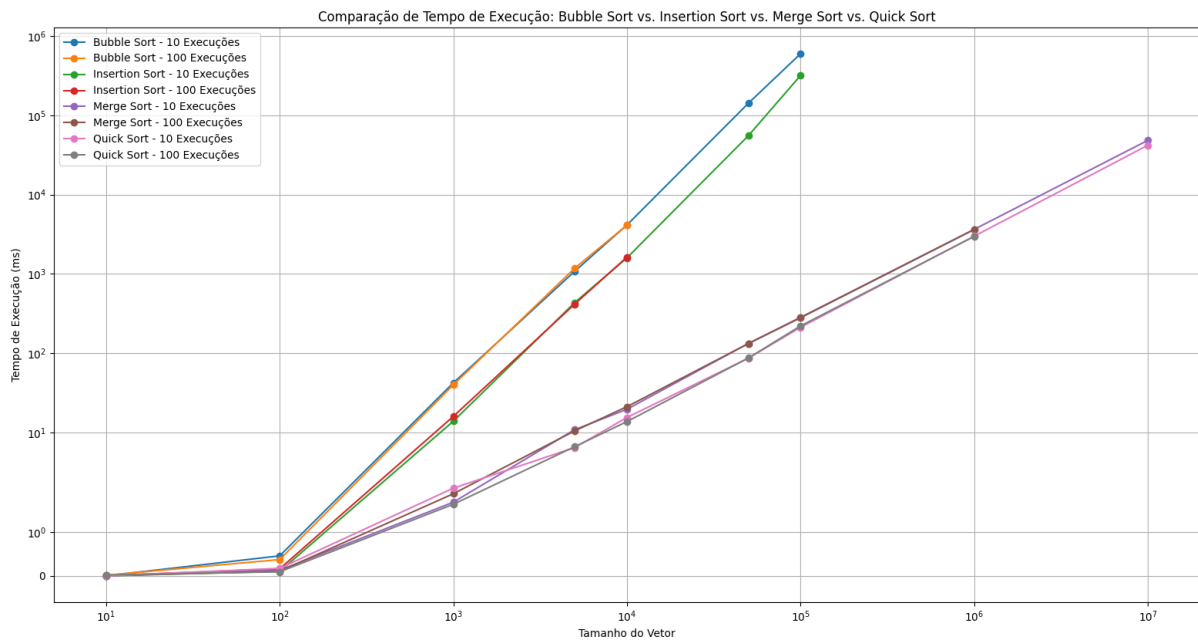
QUICK SORT

TAMANHO/ EXECUÇÕES	10^1	10^2	10^3 $5 \cdot 10^3$	10^4 $5 \cdot 10^4$	10^5	10^6	10^7
10	0.0ms	0.1703ms	1.9910ms e 6.4325ms	15.4149ms e 86.3210ms	211.6422ms	2991.3376ms	42018.2051ms
100	0.0213ms	0.0912ms	1.6245ms e 6.6007ms	13.6866ms e 86.6786ms	220.4136ms	2971.9067ms	

4 SOBRE A ANÁLISE DOS RESULTADOS

Considerando cada resultado de cada algoritmo, é evidente que os mais eficientes foram o Merge Sort e o Quick Sort, com uma pequena vantagem do Quick Sort, onde ambos possuem complexidade $\theta(n \log n)$ no melhor caso. E entre os menos eficientes, Bubble Sort e o Insertion Sort, onde os dois possuem no pior caso a complexidade $O(n^2)$, o Insertion ainda se viu com desempenho superior. Ao colocar um gráfico comparativo com as informações sobre cada algoritmo, ficou evidente que os mais demorados realmente crescem a um tempo quadrático e os mais rápidos em um tempo logarítmico. Para isso acontecer, foi necessário realizar testes com tamanhos de arrays intermediários, com 5000 e 50000 posições, para que a visualização do gráfico ficasse mais satisfatória. Fomos ainda mais longe quando analisamos um array ordenado sequencialmente para confirmar se seu tempo de execução era $\theta(n)$. E também fizemos o mesmo para o Quick Sort no pior caso, onde o pivô é sempre separação completamente desigual.

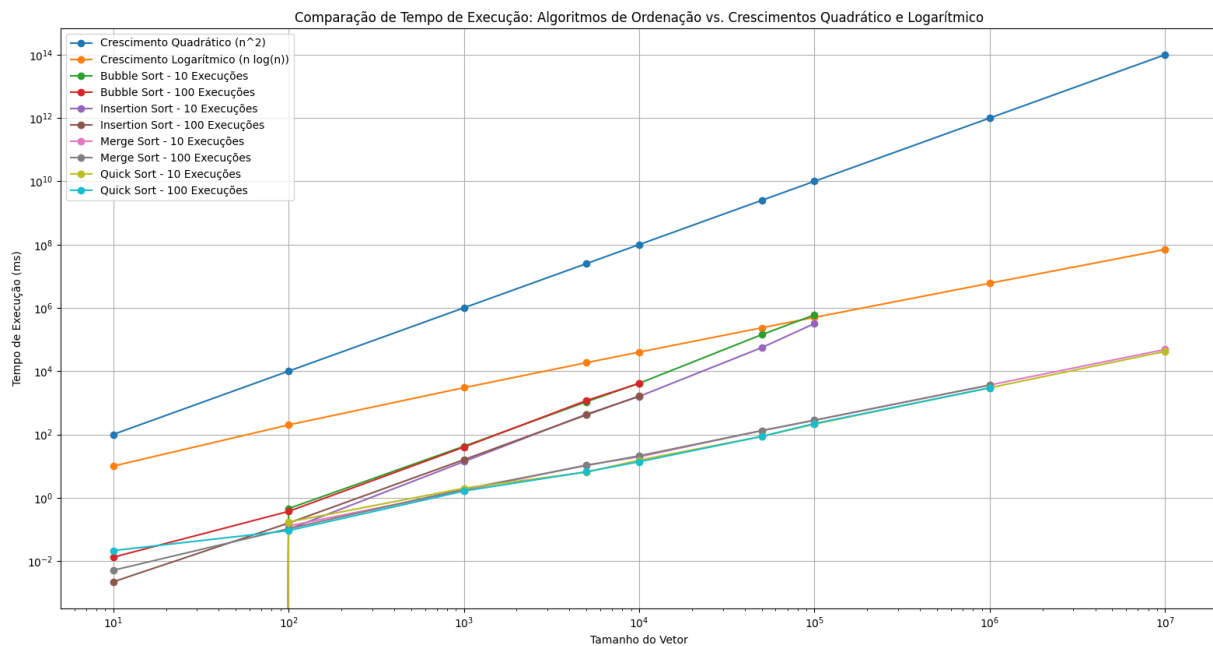
Para os algoritmos Bubble Sort e Insertion Sort, o máximo de tamanho do array conseguido para realizar os testes foi o de 50000 posições, devido ao fator do tempo ser extremamente longo para sua resolução em entradas maiores, o que não favoreceu uma maior cobertura de testes desses dois. Segue abaixo os gráficos de comparação obtidos:



O gráfico acima exibe as informações das tabelas anteriores aplicadas em um gráfico, onde para os algoritmos de complexidade $O(n^2)$, foi conseguido com arrays de 100.000 posições, e para os de complexidade $\theta(n \log n)$, conseguiu-se analisar até 10.000.000 de posições, e ao tentar mais do que isso, o computador trava e desliga sozinho devido a grande necessidade de leitura dos arquivos. Entretanto há um detalhe, a escala não estava muito clara, e foi necessário mostrar os limites de cada algoritmo, que está ilustrado no próximo gráfico.

O gráfico abaixo exibe melhor ao mostrar os limites quadráticos e logarítmicos de cada complexidade, além de exibir corretamente a presença da constante nos algoritmos mais eficientes. E ao analisar esses limites, é fácil de visualizar que o Bubble Sort e o Insertion Sort estão crescendo na mesma medida que um algoritmo de complexidade quadrática. E também é possível visualizar que os algoritmos mais eficientes crescem no ritmo exatamente descrito pelo logaritmo do tamanho da entrada multiplicado por ela. E a realização de 100 testes para um e 10 para outro, serve para evidenciar a estabilidade da medida do tempo de execução de cada um, deixado explícito na legenda.

Portanto, nota-se que sim, o comportamento prático dos algoritmos está de acordo com sua análise assintótica teórica, provando assim que a teoria também se aplica na prática.



Ao analisar a questão da existência de alguma entrada “n”, da qual os algoritmos Bubble Sort e Insertion sejam executados mais rapidamente do que os outros dois, primeiro se viu necessária a análise teórica do assunto: que trata de que esses algoritmos no melhor caso são executados em um tempo $O(n)$, dos quais aplica-se os algoritmos a um array já ordenado. Para isso foi gerado alguns arrays de tamanhos diferentes para realizar tal análise.

Ao analisar um array completamente ordenado com 1.000.000 de posições e aplicando o Bubble Sort, é obtida uma média de tempo de 61.3696ms, que é estupidamente menor do que os analisados anteriormente, do qual no array aleatório, nem sequer foi conseguido medir esse tempo sem que o computador travasse ou desligasse sozinho.

Ao analisar o mesmo array completamente ordenado com 1.000.000 de posições e aplicando o Insertion Sort, é obtida uma média de tempo de 102.9735ms, novamente muito menor do que os analisados anteriormente, e levou um tempo maior que o Bubble.

Ao analisar o mesmo array, com mesmo tamanho, porém, aplicando o Merge Sort, é obtida uma média de tempo de 2121.0092ms, muito maior do que os dois algoritmos anteriores, provando que o Merge Sort mantém seu crescimento $\theta(n \log n)$.

Ao analisar o mesmo array, com mesmo tamanho, porém, aplicando o Quick Sort, é obtida uma média de tempo de 1388.6356ms, muito maior do que os dois algoritmos anteriores, provando que o Quick Sort mantém seu crescimento $\theta(n \log n)$.

Os dois algoritmos recursivos têm uma redução em seu tempo de execução, entretanto, nada comparado aos algoritmos do Bubble Sort e Insertion Sort, que simplesmente

tem o custo linear, reduzindo drasticamente o tempo de execução. Com isso, é possível concluir que para arrays ordenados, são muito mais eficientes que os recursivos. E para entradas minúsculas, em um array de 10 elementos, o Bubble Sort e o Insertion Sort são igualmente ou até mais eficientes que os outros dois algoritmos; somente para entradas com por volta de 100 elementos é que a diferença começa a se mostrar evidente, e as medições se tornam claramente visíveis, com cada tempo indo para seus respectivos limites assintóticos.

5 SOBRE O ALGORITMO HÍBRIDO

O algoritmo híbrido implementado combina as abordagens do Merge Sort e do Insertion Sort para otimizar a ordenação de arrays grandes. O Merge Sort é um algoritmo de divisão e conquista com complexidade $\theta(n \log n)$ no melhor, pior e no caso médio. Apesar de eficiente, ele possui dois principais fatores que impactam o seu desempenho:

- **Número elevado de chamadas recursivas:** O número de chamadas recursivas cresce proporcionalmente ao tamanho da entrada, o que aumenta o custo computacional.
- **Alocação de memória para subvetores:** Em cada divisão, o Merge Sort aloca memória adicional para os subvetores, o que pode ser um gargalo para grandes volumes de dados.

A solução híbrida proposta busca minimizar esses dois problemas ao introduzir o Insertion Sort para lidar com subvetores pequenos com tamanho fixo determinado por uma constante “k” de modo que os seguintes pontos são observáveis:

- **Redução de alocações de memória:** O Insertion Sort é um algoritmo in-place (realizado diretamente no vetor principal) e não requer memória adicional para subvetores.
- **Menor custo em subvetores pequenos:** O Insertion Sort é mais eficiente para conjuntos de dados pequenos, com complexidade $\theta(n^2)$ no pior caso, mas com baixa sobrecarga devido ao tamanho reduzido dos subvetores.

Portanto, na implementação do algoritmo híbrido é mantida a estrutura principal do Merge Sort para lidar com a maior parte da ordenação, mas utiliza-se o Insertion Sort para subvetores menores. Essa integração visa reduzir o custo computacional associado às recursões

excessivas e às alocações de memória do Merge Sort puro. Assim, pode-se expressar o funcionamento em 3 etapas:

- **Divisão e Conquista:** Assim como no Merge Sort, o array é dividido recursivamente em subarrays menores. No entanto, a divisão para quando o tamanho do subarray é menor ou igual a um limite fixo k .
- **Ordenação de Subvetores Pequenos:** Para subvetores de **tamanho $\leq k$** , aplica-se o Insertion Sort, que realiza a ordenação in-place, evitando a alocação de memória adicional.
- **Combinação de Subvetores Ordenados:** Após os subvetores serem ordenados (por Insertion Sort ou Merge Sort), eles são combinados de forma eficiente usando o procedimento de merge do Merge Sort.

É importante ressaltar que o valor do limite “ k ” é fundamental para o desempenho do algoritmo híbrido. Valores muito pequenos podem não trazer ganhos significativos, enquanto valores muito grandes podem comprometer a eficiência geral ao desviar o foco do Merge Sort. Nos testes feitos com diferentes limites foi observado que quando “ k ” assume um valor entre 8 e 15, obtém-se um bom equilíbrio entre o custo do Merge Sort e a eficiência do Insertion Sort em subvetores pequenos.

6 COMPARAÇÃO ENTRE OS ALGORITMOS PUROS E O HÍBRIDO

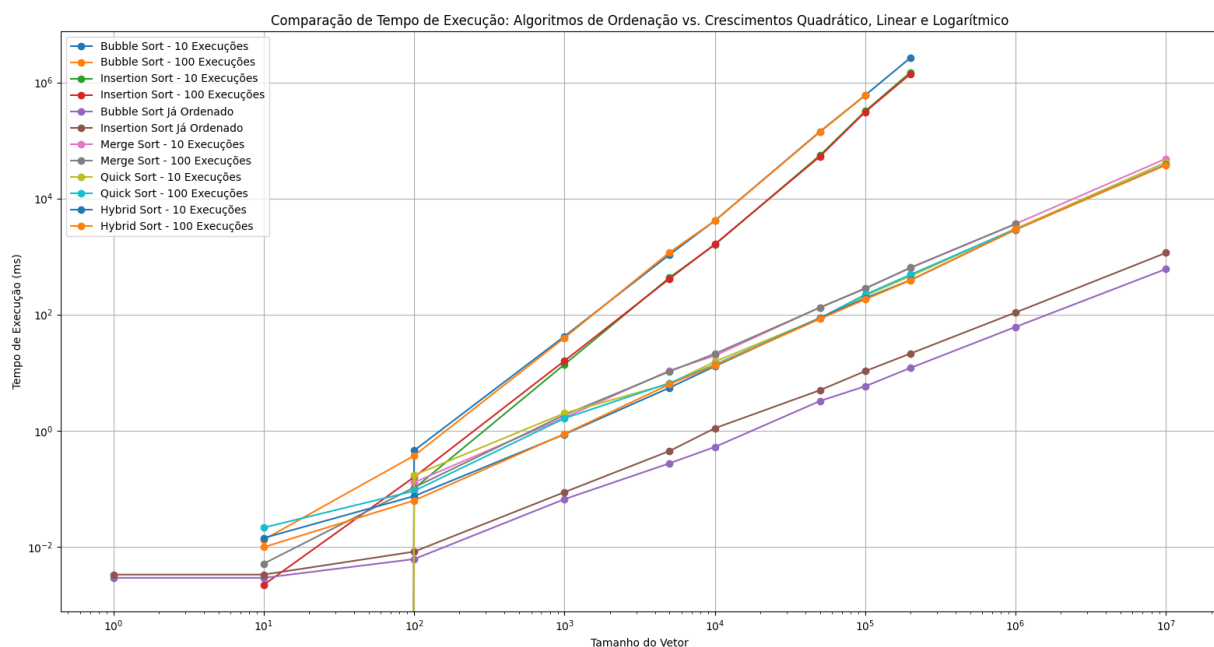
Os algoritmos puros que mostraram melhor desempenho são aqueles que no melhor caso, atingiram uma complexidade no melhor caso para $\theta(n \log n)$, para arrays aleatórios. Desconsiderando os possíveis problemas que o Quick Sort pode possuir, podemos assumir que os melhores tempos são obtidos aplicando o Quick Sort e o Merge Sort para os maiores tamanhos de entrada. O Hybrid Sort, através das análises e aferições de tempo, obteve um desempenho semelhante e por vezes até melhor do que os melhores algoritmos puros. Segue abaixo a tabela com os tempos obtidos através das médias de cada uma das aferições:

HYBRID SORT

TAMANHO/ EXECUÇÕES	10^1	10^2	10^3 $5 \cdot 10^3$	10^4 $5 \cdot 10^4$	10^5	10^6	10^7
10	0.0141ms	0.0743ms	0.8625ms e 5.4822ms	12.8021ms e 87.0905ms	191.0569ms	2897.8615ms	38352.2010ms

100	0.0098ms	0.0624ms	0.8754ms	13.2893ms	182.2432ms	2925.5070ms	37504.9145ms
			e	e			
			6.2824ms	83.8892ms			

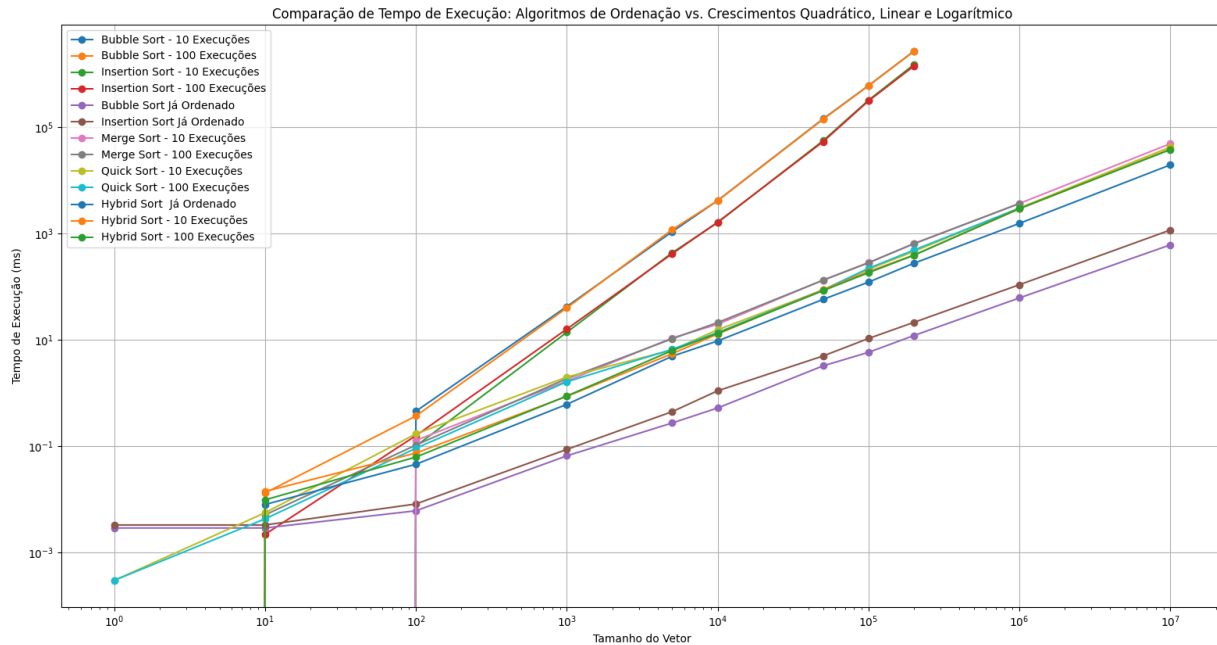
Ao analisar o tempo de execução do Hybrid Sort e ao comparar com os tempos dos outros algoritmos, fica evidente que seu crescimento é equivalente ao do Merge Sort e Quick Sort, e executa mais rapidamente que ambos, principalmente para entradas maiores, mesmo que tenha apenas um tempo um pouco melhor. O gráfico abaixo deixa evidente a pequena diferença, e ainda mostra que o crescimento ainda é logarítmico, ficando entre os algoritmos mais eficientes entre os testados.



Visualizando as cores do gráfico acima, fica visível que ao comparar os mesmos tamanhos de entrada para arrays aleatórios, o desempenho do Hybrid Sort é equivalente, e até melhor para algumas aferições realizadas, porém, visto que ele se trata de uma modificação do Merge Sort, para o pior caso, sua complexidade se manteria $\theta(n \log n)$.

Logo, conclui-se que o Hybrid Sort é mais eficiente do que o Bubble Sort e o Insertion Sort para o pior caso, sendo equivalente ao Merge Sort, para o caso médio e pior caso, e é equivalente ao Quick Sort, para o caso médio e melhor caso. Quando as regras para medição se alteram, como ao ter arrays já ordenados, os melhores tempos se dão pelo Bubble Sort e Insertion Sort, com tempo $\theta(n)$. Aplicando o Hybrid Sort em um array já ordenado, ainda é

obtido o tempo $\theta(n \log n)$, entretanto, seu desempenho acaba sendo um pouco melhor do que os outros algoritmos, como explicitado pelo gráfico a seguir:



Nota-se ao olhar a legenda, que a taxa de crescimento do Hybrid Sort se mantém na mesma medida dos outros algoritmos recursivos, mesmo o array já estando ordenado. Logo concluímos que sua notação de complexidade é $\theta(n \log n)$, tanto para o melhor quanto para o pior caso, possuindo vantagem de alguns milissegundos para cada aferição.

7 CONCLUSÃO

Os objetivos apresentados para esse trabalho foram resolvidos de forma satisfatória, e por vezes, até mesmo acima do esperado, pois todas as implementações foram feitas, explicadas e sua complexidade analisada. Os testes necessários foram feitos, tempos mensurados e ainda houve a comparação dos algoritmos puros com o híbrido. Os resultados obtidos foram condizentes com a teoria apresentada pela literatura, e por vezes melhores do que esperado. O tempo de execução altamente elevado para entradas maiores, por vezes frustrou o grupo, impedindo de executar os algoritmos não-recursivos para entradas acima de 200.000 posições. No entanto, não foi impedido de executar todos os testes para a maioria das entradas estipuladas.

Acreditamos que o trabalho foi de grande importância para relembrar os conceitos aprendidos nos semestres anteriores, e reforçar o que foi passado durante esse primeiro módulo da disciplina, fazendo entender o que foi passado pelo professor, e mesmo algumas partes, tanto

do trabalho em si, quanto deste relatório possam parecer confusas, foi feito o melhor de cada integrante para que tudo estivesse correto e aceitável, e principalmente entendível.